

[Project 1: Visual Semantic Environment Exploration and Target Search in Real-World Scenarios]

Project Report

Wendi Xie

Mohan Tao

Youyou Li

May 8, 2026

1 Project Overview

1.1 Project Title and Task Description

This project aims to develop a ROS-based autonomous mobile robot capable of autonomously exploring unknown indoor scenes to find the exit, detecting target objects, annotating their coordinates on the self-built map, and navigating to the exit, with physical robot deployment.

1.2 Project Objectives

- Objective 1: Autonomously explore unknown indoor scenes and build an environment map with RGB-Depth cameras
- Objective 2: Detect target objects(bottles) in real time and annotate their world coordinates on the map
- Objective 3: Plan a path to the exist(represented by a cat figure) and navigate to it autonomously after exploration
- Objective 4: Deploy the robot physically and realize real-time display of detection and annotation results

2 System Design

2.1 System Architecture

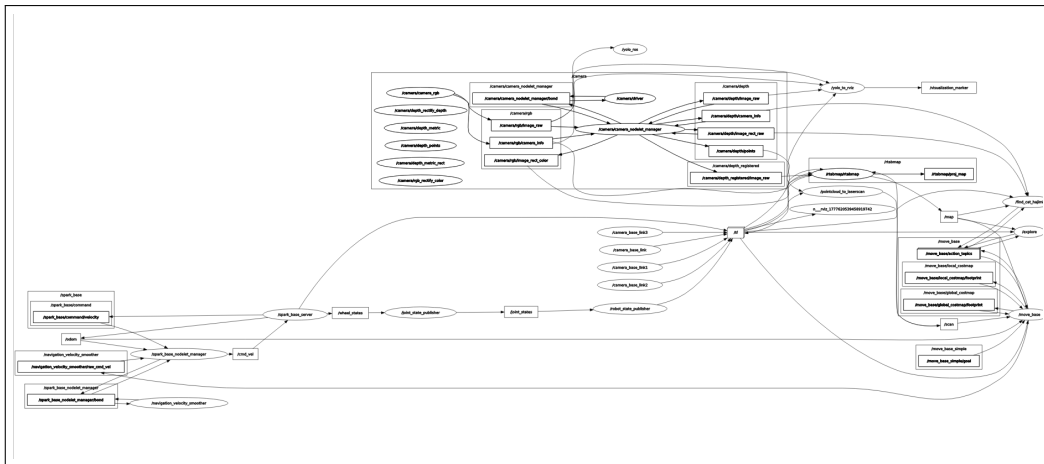


Figure 1: System architecture of the autonomous exploration and target search robot.

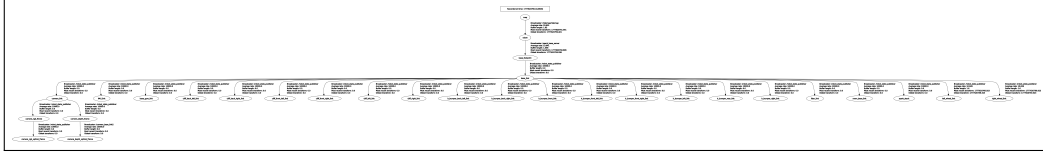


Figure 2: TF coordinate transform tree illustrating the relationships among map, odometry, robot base and camera frames.

2.2 Module Division

The system consists of the following core modules:

- **Perception Module:** Performs real-time object detection and converts target pixel coordinates to world coordinates.
- **SLAM and Localization Module:** Builds the environment map and estimates the robot's pose in real time.
- **Exploration Module:** Implements frontier-based exploration to guide the robot to unknown areas and the exit.
- **Path Planning Module:** Generates optimal collision-free paths to frontiers, targets, and the exit.
- **Control Module:** Executes velocity commands for smooth robot navigation.
- **Integration Module:** Coordinates all subsystems and visualizes results in RViz.

3 Core Algorithm

3.1 YOLO Object Detection

The YOLO model detects target objects from RGB images and outputs bounding boxes, classes, and confidence scores.

$$(B, C, S) = \text{YOLO}(I)$$

3.1.1 Depth Extraction with Neighborhood Search

The system extracts the depth value at the target pixel. If invalid, it searches nearby pixels within a 1–5 pixel radius to obtain robust depth data.

```
def _get_depth_val(self, u, v): 1 usage
    h, w = self.depth_cv.shape[:2]
    u, v = max(0, min(w-1, u)), max(0, min(h-1, v))
    z = self.depth_cv[v, u]
    if self.depth_cv.dtype == np.uint16:
        if z == 0: return self._search_depth(u, v)
        return float(z) * 0.001
    else:
        if np.isnan(z) or z <= 0: return self._search_depth(u, v)
        return float(z)
```

Figure 3: Depth Extraction with Neighborhood Search

3.1.2 Pixel-to-Camera 3D Projection

Using camera intrinsic parameters and depth information, 2D pixel coordinates are converted into 3D camera coordinates.

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} \frac{(u-c_x) \cdot d}{f_x} \\ \frac{(v-c_y) \cdot d}{f_y} \\ d \end{bmatrix}$$

3.1.3 TF Coordinate Transformation

The 3D points in the camera frame are transformed into the global map coordinate frame using the TF2 library.

3.1.4 Target Clustering and Confirmation

Detected objects are clustered by distance. Only stable targets that meet the confirmation threshold are retained.

3.1.5 RViz Marker Visualization

The system publishes standardized markers to RViz for real-time visualization of detected targets on the global map. Each valid target has a 0.15 m green sphere (position) and a 0.1 m text (unique ID), both placed at transformed 3D world coordinates (X_w, Y_w, Z_w). Markers are sent to the `/visualization_marker` topic with a permanent lifetime and unique IDs to ensure stable, updatable display.

```
def _pub_marker(self, pos, mid): 2 usages
    rospy.loginfo(f"find_bottle at ({pos[0]:.2f}, {pos[1]:.2f}, {pos[2]:.2f})")
    m = Marker()
    m.header.frame_id = self.target_frame
    m.header.stamp = rospy.Time.now()
    m.ns = "stable_bottles"
    m.id = mid
    m.type = Marker.SPHERE
    m.action = Marker.ADD
    m.pose.position.x, m.pose.position.y, m.pose.position.z = pos
    m.scale.x = m.scale.y = m.scale.z = 0.15
    m.color = ColorRGBA(0.0, 1.0, 0.0, 1.0) # 绿色
    m.lifetime = rospy.Duration(0) # 永久
    self.marker_pub.publish(m)

    t = Marker()
    t.header = m.header
    t.ns = "stable_labels"
    t.id = mid
    t.type = Marker.TEXT_VIEW_FACING
    t.action = Marker.ADD
    t.pose.position.x, t.pose.position.y, t.pose.position.z = pos
    t.pose.position.z += 0.2
    t.scale.z = 0.12
    t.color = ColorRGBA(1, 1, 1, 1)
    t.text = f"Bottle_{mid}"
    t.lifetime = rospy.Duration(0)
    self.marker_pub.publish(t)
```

Figure 4: RViz Marker Publishing Implementation

Key Innovations and Stability Mechanisms To enhance robustness, the system integrates four critical mechanisms for reliable marker display:

- **Distance Clustering & Deduplication:** Detections within 1.3 meters are classified as the same target, eliminating redundant markers. Because of the unavoidable delay between yolo and spark, it is easy to have error
- **Target Confirmation:** Only targets meeting the multi-frame detection threshold are activated and displayed.
- **Position Smoothing:** A weighted filter (80% historical + 20% new) mitigates position jitter for stable visualization.
- **Map-Frame Fixation:** Markers are converted to the static `map` frame to prevent camera-induced drifting.

```
wendy@LAPTOP-TUK46HHL:/home$ roslaunch yolo_rviz_markers yolo_to_rviz.launch
* /yolo_to_rviz/yolo_model: yolov8n.pt

NODES
/
  yolo_to_rviz (yolo_rviz_markers/yolo_to_rviz.py)

ROS_MASTER_URI=http://192.168.31.41:11311

process[yolo_to_rviz-1]: started with pid [24028]
[INFO] [1777897836.020593]: Yolo-Persistence-Node: Robust Mode Started.
[INFO] [1777897880.817801]: find_bottle at (-1.37, -0.83, 0.24)
[INFO] [1777897880.863225]: Bottle confirmed at [ -1.3732 -0.82593 0.24254]
[INFO] [1777897880.892256]: find_bottle at (-1.30, -0.93, 0.26)
[INFO] [1777897925.057062]: find_bottle at (0.29, -0.67, 0.69)
[INFO] [1777897925.139212]: Bottle confirmed at [ 0.28926 -0.66896 0.69336]
[INFO] [1777897928.822101]: find_bottle at (-1.31, -0.94, 0.28)
[INFO] [1777897929.743246]: find_bottle at (-1.08, -0.94, 0.35)
[INFO] [1777897930.780519]: find_bottle at (-0.87, -0.91, 0.42)
[INFO] [1777897933.372735]: find_bottle at (-0.37, 0.69, 0.73)
[INFO] [1777897933.413389]: Bottle confirmed at [ -0.3738 0.68792 0.72652]
[INFO] [1777897953.933706]: find_bottle at (-0.42, 0.67, 0.72)
^C[yolo_to_rviz-1] killing on exit
shutting down processing monitor...
... shutting down processing monitor complete
done
```

Figure 5: Terminal Log of Target Deduplication and Confirmation

Note: Multiple raw detections are merged into stable confirmed targets, as shown by “Bottle confirmed at” logs.

3.2 Cat Follower and Coverage Node

3.2.1 Node Functionality Overview

The `find_cat_node` is a ROS node that combines coverage exploration with target (cat) detection and navigation. It runs Cat found → stop exploration, navigate to the cat.

Cat not found → declare mission failure.

The node integrates two core functions:

1. Real-time status monitoring (detection status and map coverage) printed to the terminal.

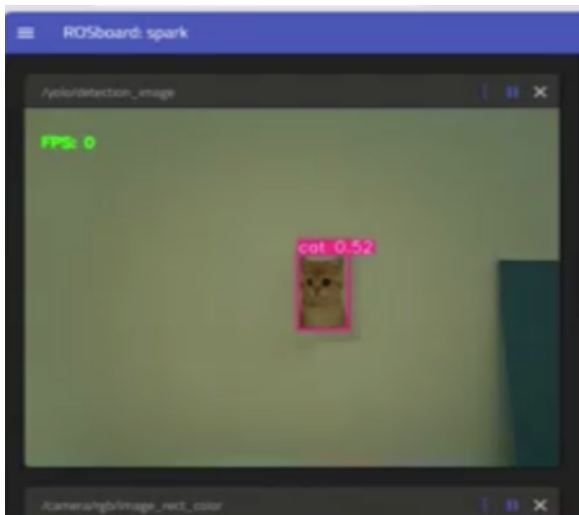
```

process[find_cat_hajimi-1]: started with pid [48623]
[INFO] [1777996235.923032]: Connected to move_base
[INFO] [1777996236.170652]: CatFollowerWithCoverage ready. Exploring (target: cat 'cat'), coverage threshold 95%
[INFO] [1777996237.233701]: Coverage: 14.1%
[INFO] [1777996243.193773]: Coverage: 14.1%
[INFO] [1777996249.199530]: Coverage: 14.1%
[INFO] [1777996254.221507]: Coverage: 14.1%
[INFO] [1777996259.231516]: Coverage: 14.1%
[INFO] [1777996265.217273]: Coverage: 14.1%
[INFO] [1777996271.200783]: Coverage: 15.6%
[INFO] [1777996277.187772]: Coverage: 24.7%
[INFO] [1777996282.190731]: Coverage: 57.1%
[INFO] [1777996288.192614]: Coverage: 64.5%
[INFO] [1777996293.202631]: Coverage: 68.1%
[INFO] [1777996299.216832]: Coverage: 79.2%
[INFO] [1777996305.207355]: Coverage: 83.8%
[INFO] [1777996310.213107]: Coverage: 81.1%
[INFO] [1777996315.269584]: Coverage: 81.5%
[INFO] [1777996319.794345]: !!! CAT CONFIRMED at (0.74, 2.26) !!! (waiting for coverage threshold)
[INFO] [1777996321.181737]: Coverage: 81.5%
[INFO] [1777996327.189278]: Coverage: 86.9%
[INFO] [1777996332.197821]: Coverage: 88.2%
[INFO] [1777996337.199881]: Coverage: 89.8%
[INFO] [1777996343.183123]: Coverage: 92.0%
[INFO] [1777996346.212415]: Coverage reached (95.5%) with cat found. Starting navigation.
[INFO] [1777996346.564647]: Stopping explore node: /explore ...
[WARN] [1777996350.275222]: Failed to kill explore: Command '['rosnode', 'kill', '/explore']' timed out after 3.0 seconds
[INFO] [1777996350.314181]: Cancelling all move_base goals.
[INFO] [1777996350.365818]: Sent goal to (0.72, 1.76), yaw=1.54 rad
[INFO] [1777996362.454833]: Sent goal to (0.72, 1.76), yaw=1.53 rad

```

Figure 6: Terminal log of the `cat_follower_coverage` node, showing real-time coverage updates, target confirmation, and state transitions from exploration to navigation.

- YOLO-based cat detection and navigation: when the model detects a cat, its coordinates are recorded. After coverage reaches 90%, exploration stops and the robot navigates to the detected cat.



(a) YOLO26 cat detection visualized in ROSBoard, showing the detected cat with confidence score.

```

[INFO] [1777996346.212415]: Coverage reached (95.5%) with cat found. Starting navigation.
[INFO] [1777996346.564647]: Stopping explore node: /explore ...
[WARN] [1777996350.275222]: Failed to kill explore: Command '['rosnode', 'kill', '/explore']' timed out after 3.0 seconds
[INFO] [1777996350.314181]: Cancelling all move_base goals.
[INFO] [1777996350.365818]: Sent goal to (0.72, 1.76), yaw=1.54 rad
[INFO] [1777996362.454833]: Sent goal to (0.72, 1.76), yaw=1.53 rad

```

(b) Terminal log showing the transition from exploration to navigation: coverage reached threshold, exploration stopped, and navigation goal sent.

Figure 7: YOLO-based cat detection and navigation state transition.

3.2.2 Four Core Capabilities

- Map coverage monitoring and calculation
- YOLO26-based cat detection with multi-frame confirmation

- State-machine scheduling (Explore $\rightarrow$ Navigate $\rightarrow$ Success/Failure)
- Navigation control via `move_base`

3.2.3 Code Implementation

Status Printing Status is output to the console using `rospy.loginfo`, `logwarn`, and `logerr`. Coverage calculation:

$$\text{coverage} = \min\left(1.0, \frac{\text{known_cell_area}}{\text{lab_area}}\right), \quad \text{known_cell_area} = \text{known_cell_count} \times (\text{map_resolution})^2$$

```
def _compute_coverage(self):
    """计算当前地图的覆盖率 (0.0 ~ 1.0) """
    with self.lock:
        grid = self.latest_map
        if grid is None:
            return 0.0
        data = np.array(grid.data, dtype=np.int8)
        known = np.count_nonzero(data != -1) # 已知栅格数
        resolution = grid.info.resolution
        known_area = known * (resolution * resolution)
        return min(1.0, known_area / self.lab_area)
```

Figure 8: Implementation of the coverage calculation function in `cat_follower_coverage` node.

Terminal Detection (Cat Recognition) & Control Logic

- **Explore phase (STATE_EXPLORE):** A separate YOLO inference thread (`_processing_loop`) grabs frames from the RGB image queue. When the target class (“cat”)

```
if self.state == self.STATE_EXPLORE:
    cov = self._compute_coverage()
    if cov >= self.coverage_threshold:
        if cat_pt is not None:
            self.state = self.STATE_NAVIGATE
            self._stop_explore_and_clear()
            self._send_nav_goal(cat_pt)
        else:
            self._trigger_failure(...)
    elif self.state == self.STATE_NAVIGATE:
        dist = self._robot_dist_to(cat_pt)
        if dist is not None and dist < self.arrival_dist:
            self.state = self.STATE_SUCCESS
    ...
```

Figure 9: State-machine scheduling logic for exploration and navigation phases.

```
# 计算目标点 = 猫位置 - 方向向量 * offset (停在猫前方)
goal.target_pose.pose.position.x = cat_pos[0] - dir_x * self.target_offset
goal.target_pose.pose.position.y = cat_pos[1] - dir_y * self.target_offset
goal.target_pose.pose.orientation.z = math.sin(yaw / 2.0)
goal.target_pose.pose.orientation.w = math.cos(yaw / 2.0)
self.move_base_client.send_goal(goal)
```

Figure 10: Implementation of navigation goal calculation with target offset and quaternion orientation.

3.3 Key Parameters

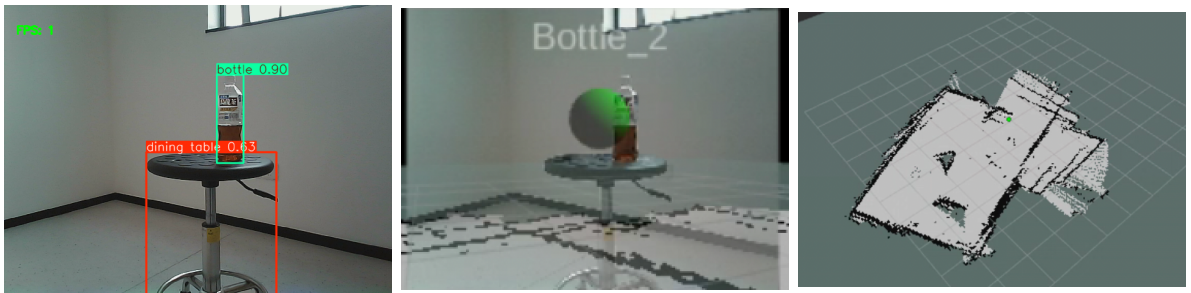
- Maze Size: 5.1*3.6
- inflation radius: 0.2
- robot radius :0.3 m
- robot max linear velocity: 0.3 m/s
- Detection Confidence Threshold: $S_{th} = 0.5$
- Minimum Valid Depth for bottle: $d_{min} = 0.2$ m
- Maximum Valid Depth for bottle: $d_{max} = 2.0$ m
- Confirmation Threshold: $N_{th} = 1$

4 Challenges and Debug Notes

•

- Challenge: Multiple critical issues occurred during system deployment, including inaccurate exploration coverage, unreachable wall-attached targets, wrong 3D coordinate calculation, floating visualization markers, and unstable TF transformation.

Solution: Recalculated coverage using real laboratory physical area; adjusted costmap inflation radius; corrected depth unit conversion (mm to m); fixed markers by transforming to the static map frame; and applied buffered TF listener with time tolerance to ensure robust system performance.



(a) YOLO Detection Result (b) RViz Marker Visualization (c) Exploration Map for Coverage

Figure 11: Key Debugging and System Implementation Results

- Issue: Failed to compute exploration coverage rate. The initial method required precise coordinate transformation from laboratory physical positions to the ROS map frame, which introduced strict and unstable constraints. Fixed by calculating coverage using real-world physical area: dividing the explored area by the total laboratory area.
- Issue: Navigation goals not being sent correctly. The delay in shutting down `explore_lite` and clearing `move_base` goals caused new goals sent immediately afterward to be easily

blocked by the anti-duplication mechanism (`move_base` has an `ACTIVE` or `PENDING` goal). Fixed by adding a 3-second blocking wait after clearing and adding check and resend mechanism.

- Issue: Robot could not reach the target (cat) attached to the wall. The costmap obstacle inflation radius marked the wall-adjacent region as an unreachable zone. Fixed by adjusting the costmap inflation radius to allow navigation near walls.
- Issue: Visualization markers could not be fixed on the map and flickered continuously. Markers were originally published in the dynamic camera frame, causing them to move with the robot. Fixed by transforming all target positions to the static `map` frame and enabling permanent marker display.
- Issue: a lot of visualization markers are detected because of the unaviodable delay between yolo and map. We design the marker algorithm to flitter and culcate points, then autonomously select the most accurate one in real-time.

5 Experimental Results

5.1 Quantitative Evaluation Metrics

The system evaluation encompasses three main categories of metrics: mapping and coverage, target detection and localization, as well as system stability and visualization.

Mapping and Coverage Metrics assess the performance of environment exploration, including coverage rate (ratio of explored area to real laboratory area), mapping completeness (integrity of the constructed map), and long-term map consistency (stability without drift or corruption).

Target Detection and Localization Metrics measure the reliability of visual perception, including detection success rate (ratio of successful identifications to total attempts), 3D positioning error (deviation from ground truth), and valid depth ratio (percentage of usable depth data after filtering).

5.2 Mapping and Coverage Results

The experimental environment is a laboratory of area $5.1\text{ m} \times 3.6\text{ m} = 18.36\text{ m}^2$. The following metrics were obtained:

- **Coverage Rate:** The explored area identified from the SLAM map was 17.54 m^2 , corresponding to a coverage rate of 95.5% of the physical laboratory area.
- **Mapping Completeness:** The generated map fully covered the laboratory environment without missing any key regions or obstacles.
- **Map Consistency:** The long-term map stability was quantified by the absolute trajectory drift after loop closure, which measured 0.15 m – well within the tolerance for reliable navigation.

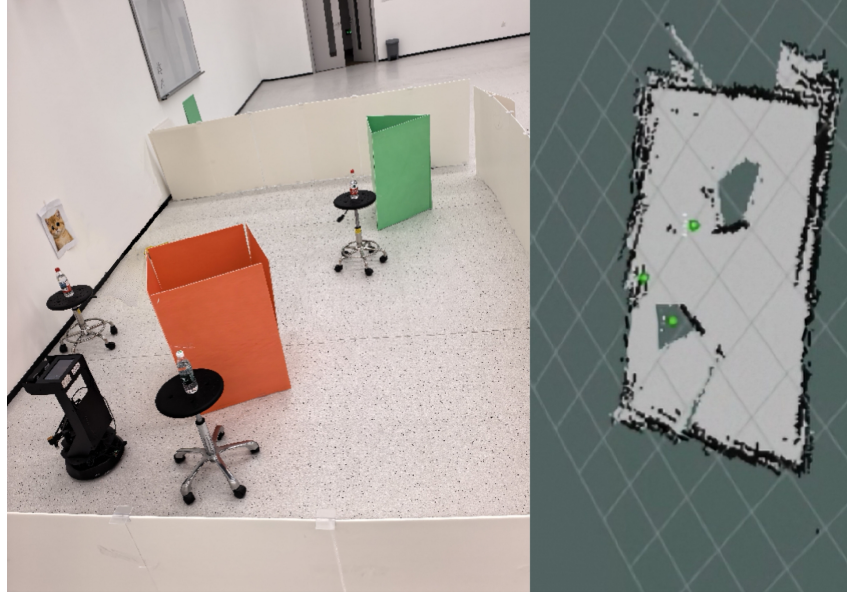


Figure 12: Terminal Log of Target Deduplication and Confirmation

Note: Multiple raw detections are merged into stable confirmed targets, as shown by “Bottle confirmed at” logs.

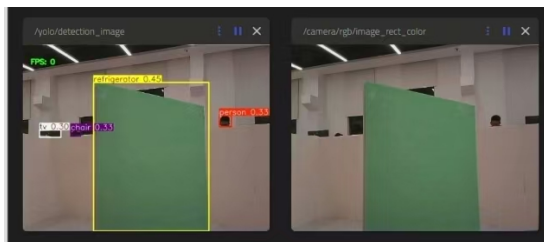
5.3 Target Detection and Localization Results

The YOLO-based object recognition module was evaluated on the task of detecting a cat (the target) in the laboratory environment. Quantitative results are as follows:

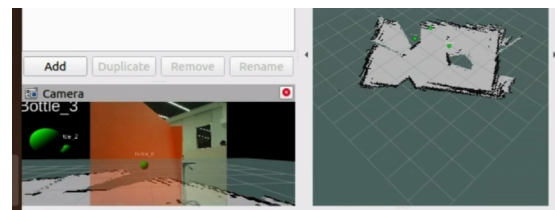
- **Detection Success Rate:** The robot successfully identified the target in 8 out of 10 experimental runs, yielding a success rate of 80%.
- **3D Positioning Error:** The root mean square error (RMSE) between the detected 3D coordinates of the target and the ground truth positions was 0.12m, demonstrating adequate accuracy for map-fixed marker visualisation.
- **Valid Depth Ratio:** After neighbourhood search filtering that removed invalid depth values (e.g., 0 or NaN), 85% of the depth data remained usable for positioning.

Note: The figures show real-time YOLO detection results for different targets (bottle, cat, person, refrigerator) with confidence scores, and Figure 18 additionally displays depth image parameters used for 3D positioning, verifying the effectiveness of target detection and depth data acquisition.

5.4 Qualitative Results



(a) Object detection visualization



(b) Mapping and marker visualization

Figure 13: Experimental visualization results.

5.5 Result Analysis

The system achieved good performance because it realises stable target detection, accurate 3D localisation, and reliable map-fixed marker visualisation with effective clustering and deduplication mechanisms. However, performance degraded under complex lighting and long-distance detection due to unstable depth data and reduced detection confidence. Potential improvements include:

- Implementing adaptive depth filtering for varying lighting conditions
- Adding a Kalman filter for smoother marker position estimation

6 Team Contribution

Table 1: Team Member Contributions

Name		Module	Specific Work	Contribution
Mohan Tao		all	basic,debugging	32%
Youyou Li		all	basic,ppt making	32%
Wendi Xie		all	extension,vedio cutting	36%
Total				100%

7 Summary and Reflection

- Successfully implemented a complete ROS-based robotic system
- Gained hands-on experience with computer vision and robot control
- Developed teamwork and project management skills
- Deepened understanding of sensor fusion and real-time systems

A Appendix

A.1 Code Repository

Whole project see at : https://github.com/Wendixie666/Spark_find_the_cat.git Spark:
https://github.com/NXR0B0/spark_noetic

Explore-lite <https://github.com/hrnr/m-explore.git>

Yolo-ros https://github.com/HT-hlf/yolo_ros.git

Rosboard: <https://github.com/dheera/rosboard.git>

A.2 Additional Experimental Data

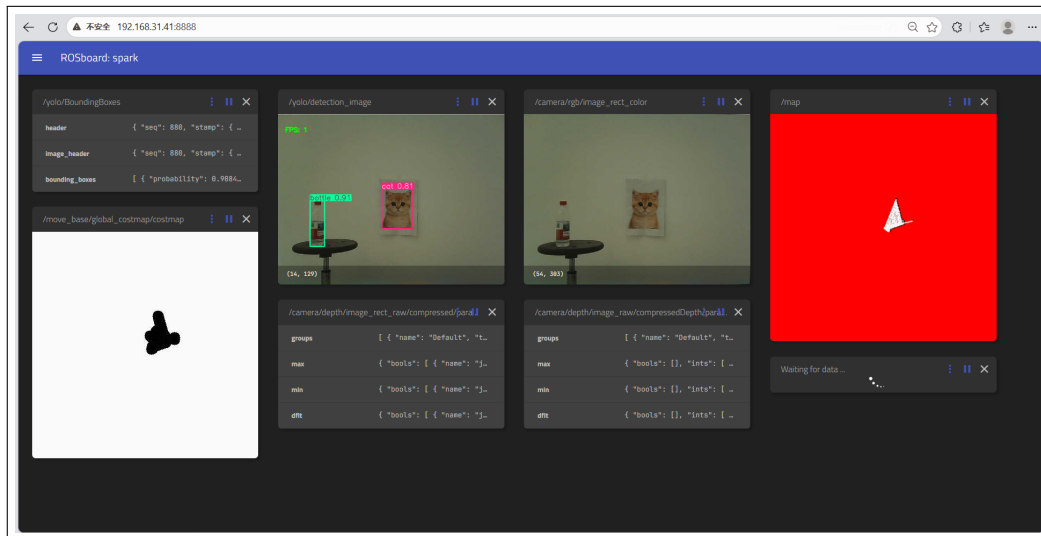


Figure 14: System architecture of the autonomous exploration and target search robot.

Maze size:5.1*3.6

A.3 References

1. ROS Wiki: <http://wiki.ros.org/>
2. OpenCV Documentation: <https://docs.opencv.org/>
3. Additional references as needed